Programación Orientada a Objetos en Python

La Programación Orientada a Objetos (POO) es un paradigma de programación que organiza el software en torno a objetos, los cuales combinan estado (datos) y comportamiento (funciones). Este paradigma permite modelar problemas del mundo real de forma más natural, favoreciendo la modularidad, la reutilización de código y el mantenimiento del software.
En Python, como cualquier otro lenguaje orientado a objetos, la POO se basa principalmente en el uso de clases y objetos.

1 CLASES Y OBJETOS

Una clase es una abstracción que define un tipo de objeto. Describe:

- **Atributos**: variables asociadas a los objetos de esa clase.
- **Métodos**: funciones que definen el comportamiento de los objetos.

Un objeto es una instancia concreta de una clase, creada a partir de dicha definición. Por ejemplo, si tuviéramos la clase Gato, sus características serían declaradas como atributos, que son variables que pertenecen a la clase. Por otro lado, las clases tienen funcionalidades, en este caso, el Gato podría tener la funcionalidad maullar o ronronear. A estas funcionalidades se las denominará métodos.
Las clases son la idea de Gato, pero no es un Gato, un gato es el ser vivo que tenemos en casa, sentado encima de nuestros ejercicios de programación en Python. Tenemos la idea de gato, aquello que nos hace saber que eso que está maullando en busca de comida es un gato, pero con las ideas no podemos trabajar. Por este motivo, existen los objetos, los objetos son instancias de una clase, la representación de dicha idea, con la cual sí que podemos trabajar. Para crear un objeto, a partir de una clase, necesitaremos un método constructor.

```python
class Gato:
    def __init__(self, nombre, raza, color_pelaje, sexo):
        self.nombre = nombre
        self.raza = raza
        self.color_pelaje = color_pelaje
        self.sexo = sexo

    def maullar(self):
        print("miau")

    def ronronear(self):
        print("brrrrr")

    def cambiar_pelaje(self, nuevo_pelaje):
        self.color_pelaje = nuevo_pelaje
```

1.1 CONSTRUCTOR

El método `__init__` es el constructor de la clase. Se ejecuta automáticamente al crear un objeto y se utiliza para inicializar su estado.
El parámetro `self` es una referencia al objeto que está ejecutando el método. Permite acceder a sus atributos y a otros métodos de la misma instancia.

2 INSTANCIACIÓN Y USO DE OBJETOS

```python
misifu = Gato("Misifú", "gato común europeo", "negro", "macho")

misifu.maullar()
misifu.ronronear()

print(misifu.nombre)
print(type(misifu))
```

Conceptualmente: `misifu` es un objeto de la clase `Gato` y cada objeto mantiene su propio estado.

3 TIPOS DE MÉTODOS EN PYTHON

3.1 MÉTODOS DE INSTANCIA

Los métodos de instancia son los métodos que pertenecen al objeto creado cuando instanciamos la clase. Los métodos maullar y ronronear son métodos de instancia. Pueden acceder y modificar atributos de los objetos. Acceder a métodos y modificar el estado de la clase.

```
def maullar(self):
    print("miau")
```

3.2 MÉTODOS DE CLASE

Los métodos de clase operan sobre la clase, no sobre una instancia concreta. Reciben como primer parámetro `cls`, que representa la clase. Se declaran con el decorador `@classmethod`. Estos métodos son útiles para modificar atributos de clase y definir comportamientos comunes a todas las instancias.

3.3 MÉTODOS ESTÁTICOS

Los métodos estáticos no reciben ni la instancia ni la clase. No pueden modificar el estado del objeto ni de la clase. Se utilizan cuando la funcionalidad está relacionado conceptualmente con la clase, pero no depende de su estado. Se declaran con el decorador `@staticmethod`

```
class Calculadora:
    @staticmethod
    def sum(n1, n2):
        return n1 + n2
```

!! Los métodos estáticos no puede acceder a los atributos de las clases.

```python
class Calculadora:
    num_calculos = 0

    @classmethod
    def suma(cls, n1, n2):
        cls.num_calculos += 1
        return n1 + n2

    def sumar(self, n1, n2):
        self.num_calculos += 1
        return n1 + n2

    @staticmethod
    def sum(n1, n2):
        return n1 + n2

print("Estado inicial")
print(f"class: {Calculadora.num_calculos}")
Calculadora.suma(1, 1)
print("Estado tras sumar desde el método de clase")
print(f"class: {Calculadora.num_calculos}")
print("Estado tras crear una instancia y sumar desde el método de
↪  instancia")
calc = Calculadora()
calc.sumar(1,1)
print(f"class {Calculadora.num_calculos}")
print(f"inst {calc.num_calculos}")
print("Estado tras (otra vez) sumar desde el método de clase")
Calculadora.suma(1, 1)
print(f"class {Calculadora.num_calculos}")
print(f"inst {calc.num_calculos}")
print("Estado tras sumar (otra vez) desde el método de instancia")
calc.sumar(1,1)
print(f"class {Calculadora.num_calculos}")
print(f"inst {calc.num_calculos}")
```

4 ATRIBUTOS DE CLASE VS. ATRIBUTOS DE INSTANCIA

Existen dos conceptos a la hora de hablar de atributos:

- **Atributos de clase**: compartidos por todas las instancias.
- **Atributos de instancia**: propios de cada objeto

Modificar un atributo de clase desde una instancia puede crear un atributo nuevo solo en esa instancia, lo cual es importante entender para evitar errores.

```
class Ejemplo:
    atributo_clase = 0

    def __init__(self):
        self.atributo_instancia = 0
```

5 HERENCIA

La herencia es un proceso mediante el cual se puede crear una clase hija que hereda de una clase madre, compartiendo sus métodos y atributos. Además, una clase hija puede sobrescribir los métodos o atributos, e incluso definir unos nuevos.

Se puede crear una clase hija con tan solo pasar por parámetro la clase que queremos heredar. A modo de ejemplo, vamos a crear una clase `Animal` de la que van a heredar dos clases, reutilizando atributos y métodos, que van a ser la clase `Gato` y la clase `Perro`.

```
class Animal:
    def __init__(self, especie):
        self.especie = especie

    def hablar(self):
        print(f"Ruido de clase {type(self)} de la especie {self.especie}")
```

```
class Gato(Animal):
    def __init__(self, pelaje):
        super().__init__("gato")
        self.pelaje = pelaje

    def hablar(self):
        print("miau")
```

```python
class Perro(Animal):
    def __init__(self, pelaje):
        super().__init__("perro")
        self.pelaje = pelaje

    def hablar(self):
        print("guau")
```

Dado que una clase hija hereda los atributos y métodos de la clase madre, nos puede resultar muy útil cuando tengamos clases que comparten atributos y/o métodos pero que se diferencian en sutilezas.
Por ejemplo, tanto los perros como los gatos coinciden en que tienen un método para comunicarse y que son de una determinada especie, por lo tanto podemos abstraer cierta información en una clase madre para ambas. En este caso ha sido animal, pero podría haber sido Mamifero, por ejemplo.

!! Realizar abstracciones y buscar el común denominador para definir una clase de la que hereden las demás, es una tarea compleja, peor que puede favorecer a ahorrar código y no repetirte. Esta lógica de intentar reducir el código que se repite es la filosofía **DRY** (*Don't Repeat Yourself*).

6 POLIMORFISMO

El polimorfismo permite que objetos de distintas clases respondan de forma diferente al mismo método.

```python
animales = [Gato("blanco"), Perro("negro"), Animal("vaca")]

for animal in animales:
    animal.hablar()
```

Aunque todos los objetos comparten el método `hablar`, el comportamiento depende de la clase concreta del objeto.

7 ENCAPSULACIÓN

La encapsulación consiste en controlar el acceso al estado interno de los objetos. En Python, la encapsulación se basa en convenciones:

```
self._atributo
```

!! Python confía en que se va a respetar las convenciones y que si se encuentra un atributo precedido de un guion bajo (_), ese atributo no debe de ser utilizado, puesto que es un atributo *privado*.[1]

8 DUNDER METHODS

Existen una serie de métodos con nombres reservados que comienzan y terminan con doble guion bajo, por ejemplo:

```
__init__
__str__
__eq__
```

!! Se conocen informalmente como dunder methods (*double underscore*).

Estos métodos no se llaman normalmente de forma directa, sino que son invocados automáticamente por el intérprete de Python en situaciones concretas. Permiten definir el comportamiento de los objetos frente a operaciones del lenguaje. Algunos de los más utilizados son:

- **__init__**. Inicialización del objeto. Se ejecuta automáticamente al crear un objeto y sirve para inicializar su estado.

```
class Gato:
    def __init__(self, nombre, color):
        self.nombre = nombre
        self.color = color
```

[1] En otros lenguajes más restrictivos como Java se utiliza la cláusula `private` para evitar que nadie pueda acceder a este atributo método.

- **__str__**. Representación "legible" del objeto. El método __str__ define cómo se muestra un objeto cuando se convierte a cadena, normalmente para el usuario final. Se utiliza cuando se llama a `print(objeto)` o `str(objeto)`.

```python
class Gato:
    def __init__(self, nombre, color):
        self.nombre = nombre
        self.color = color

    def __str__(self):
        return f"Gato llamado {self.nombre}, color {self.color}"
```

!! __str__ debe devolver un `str`, nunca imprimir.

- **__repr__**. Representación "técnica" del objeto. El método __repr__ define una representación precisa y no ambigua, orientada al desarrollador. Se usa cuando el objeto aparece dentro de una colección o se escribe en una consola interactiva.

```python
class Gato:
    def __init__(self, nombre, color):
        self.nombre = nombre
        self.color = color

    def __repr__(self):
        return f"Gato(nombre='{self.nombre}', color='{self.color}')"
```

!! __repr__ debería devolver una cadena que permita recrear el objeto.

- **__len__**. Define el comportamiento de `len(objeto)`

```python
class Grupo:
    def __init__(self, alumnos):
        self.alumnos = alumnos

    def __len__(self):
        return len(self.alumnos)
```

```
g = Grupo(["Renoir", "Verso", "Alicia"])
len(g)  # Devuelve 3
```

- **__eq__**. Define qué significa que dos objetos sean "iguales".

```
class Punto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, otro):
        return self.x == otro.x and self.y == otro.y
```

```
p1 = Punto(1, 2)
p2 = Punto(1, 2)
p1 == p2
```

- **__lt__** o **__gt__**. Define como comparar objetos permitiendo ordenar objetos, son utilizados por `sorted()` o `>` `<` `>=` `<=`

```
class Punto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __lt__(self, otro):
        return self.x < otro.x

    def __gt__(self, otro):
        return self.y > otro.y
```

- **__add__**. Permite definir el comportamiento del operador +.

```
class Vector:
    def __init__(self, x):
        self.x = x

    def __add__(self, otro):
        return Vector(self.x + otro.x)
```